

AIChE Platform

Frontend Integration Guide

A comprehensive guide for frontend developers
to integrate with the AIChE Backend API

Version 1.0 - September 2026

Web Development Committee - AIChE Cairo University Student Chapter

Table of Contents

- 1. System Overview & Introduction
- 2. Technology Stack
- 3. Roles, Committees & Permissions
- 4. Admin Accounts & Default Credentials
- 5. Authentication Flow (JWT)
- 6. API Endpoints - Events & Site Visits
- 7. API Endpoints - Committee Sessions
- 8. API Endpoints - Courses & Learning Track
- 9. API Endpoints - Workshops
- 10. API Endpoints - Certificates
- 11. API Endpoints - Points & Leaderboard
- 12. API Endpoints - Feedback & Reports
- 13. API Endpoints - Dashboard & Access Control
- 14. Certificate Generation Logic
- 15. Error Handling Guide
- 16. Development Tips for Frontend

1. System Overview & Introduction

The AIChE Platform is a full-stack web application designed to automate and manage the operations of the AIChE Cairo University Student Chapter. It centralizes point tracking, certificate generation, event management, course/workshop enrollment, session grading, and internal performance reviews.

The backend is built with FastAPI (Python) and uses a PostgreSQL database hosted on Neon. It uses stateless JWT tokens for authentication, and integrates with Google Drive and Cloudinary for cloud storage of dynamically generated certificates.

Core Features

- **Dynamic Certificate Engine:** Automatically generates personalized PNG certificates with the user's name and the event/course/session title printed onto a template. Certificates are uploaded to the cloud and a public URL is stored.
- **Automated Point Tracking:** Points are never manually edited. They are strictly added/subtracted as ledger entries whenever a user completes a course, attends an event, receives QC deductions, etc.
- **Strict Permission Hierarchy:** Role-based access control ensures committee heads can only manage their own members, while the Executive Board maintains full oversight.
- **Sequential Course Unlock:** Course parts must be completed in order. Part N+1 cannot be opened until Part N is marked as completed.

2. Technology Stack

Backend

- Framework: FastAPI (Python 3.11+)
- Database: PostgreSQL (hosted on Neon)
- ORM: SQLAlchemy 2.0+
- Migrations: Alembic
- Authentication: JWT via python-jose
- Password Hashing: passlib with bcrypt
- Cloud Storage: Google Drive API + Cloudinary
- Image Generation: Pillow (PIL)

API Documentation

FastAPI automatically generates interactive API documentation:

- Swagger UI: <http://localhost:8000/docs>
- ReDoc: <http://localhost:8000/redoc>

Running the Backend Locally

```
cd backend
..\venv\Scripts\activate
uvicorn app.main:app --reload
```

The server will start at <http://localhost:8000>. All API routes are prefixed with `/api/`.

3. Roles, Committees & Permissions

The Executive Board (President, Vice President, Secretary)

- Full access to everything in the system.
- Can create/delete any event, course, workshop, or session.
- Can view all sessions from every committee (no visibility restrictions).
- Can edit any user, approve/reject feedback reports.
- Can permanently delete user accounts based on QC flags.
- Can see the full points leaderboard without restrictions.
- Can manually issue certificates to any user.

QC (Quality Control) Committee

- Global Oversight role - can view sessions and members of ALL committees.
- Cannot delete users directly, but can flag feedback for review.
- Issues point deductions for poor attendance or performance across the organization.
- Has read-only cross-committee visibility that no other committee has.

Committee Admins (e.g., OC Head, PR Head, Tech Head)

- Restricted strictly to their own committee, plus public resources.
- Can create and manage sessions for their own committee only.
- Can take attendance and track task completion for their members.
- Can issue manual certificates for their members.
- CANNOT view the internal sessions of other committees.

General Members

- Can register for events and site visits.
- Can enroll in courses and complete workshop sessions.
- Can earn certificates upon completing courses, workshops, events, or sessions.
- Can submit feedback on sessions.
- Can only view sessions marked as 'public' or belonging to their assigned committee.

4. Admin Accounts & Default Credentials

The following accounts are pre-seeded into the database when running seed.py. Frontend developers can use these credentials for testing and development. All passwords should be changed before going to production.

Role	Full Name	Username	Email	Password
President	Head President	president	president@aiiche.com	ChangeMe123!
Comm. Admin	Marketing Head	marketing_head	AICHECU.Marketing@gmail.com	AICHECU.Marketing@2026
Comm. Admin	OC Head	oc_head	AICHECU.OC@gmail.com	AICHECU.OC@2026
Comm. Admin	Technical Head	technical_head	AICHECU.Technical@gmail.com	AICHECU.Technical@2026
Comm. Admin	WebDev Head	webdev_head	AICHECU.WebDevelopment@gmail.com	AICHECU.WebDevelopment@2026
Comm. Admin	PR Head	pr_head	aichecu.pr@gmail.com	aichecu.pr@2026
Comm. Admin	QC Head	qc_head	aiche.cu.qc@gmail.com	aiche.cu.qc@2026
Comm. Admin	General Head	general_head	AICHECUSCgeneral@gmail.com	AICHECUSCgeneral@2026

Note: The default password pattern for committee admins is: {email_prefix}@2026

5. Authentication Flow (JWT)

The API uses OAuth2 with Password Flow. All protected endpoints require a valid JWT token in the Authorization header.

Step 1: Login

Send a POST request to `/api/auth/login` with username and password as x-www-form-urlencoded form data (NOT JSON).

```
POST /api/auth/login
Content-Type: application/x-www-form-urlencoded

username=president&password=ChangeMe123!
```

Step 2: Receive Token

The server returns a JSON response containing the `access_token`:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "bearer"
}
```

Step 3: Use Token in Requests

For ALL protected routes, attach the token in the Authorization header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Step 4: Get Current User Info

```
GET /api/auth/me
Authorization: Bearer <token>
```

Returns the full profile of the currently logged-in user, including their role, committee, and permissions.

Token Expiry

Tokens expire after 1444 minutes (~24 hours). After expiry, the frontend should redirect the user to the login page.

6. API Endpoints - Events & Site Visits

Base path: /api/events

GET /api/events/ - List all events

- Auth: Public (no token required)
- Returns all events/site visits with a computed 'status' field (upcoming / closed_registration / finished).
- Status is calculated in real-time from the event date and registration_deadline (never stored).

POST /api/events/ - Create an event

- Auth: committee_admin, president, vice_president, secretary
- Body: { type, title, description, date, location, registration_deadline }
- All datetimes are normalized to UTC+3.

POST /api/events/{event_id}/register - Register for event

- Auth: Any logged-in user
- Automatically blocked with 400 if registration_deadline has passed.
- Duplicate registration returns 400.

POST /api/events/{event_id}/attendance - Mark attendance

- Auth: Admin roles only
- Body: { user_id, status } where status is 'attended', 'absent_excused', or 'absent_unexcused'
- Points are automatically awarded: +2 for attended, 0 for excused, -1 for unexcused.

7. API Endpoints - Committee Sessions

Base path: /api/sessions

GET /api/sessions/ - List visible sessions

- Auth: Any logged-in user
- IMPORTANT: The backend automatically filters results based on the user's role:
 - QC admin + President/VP/Secretary: see ALL sessions from every committee.
 - Regular members/admins: see only their own committee's sessions + public sessions.

POST /api/sessions/ - Create a session

- Auth: committee_admin, president, vice_president, secretary
- Body: { committee_id, title, description, date, visibility, status }
- visibility: 'public' or 'committee_specific'

POST /api/sessions/{session_id}/scoring - Grade a member

- Auth: Admin roles only
- Body: { user_id, attendance_status, task_status, engagement_status, bonus_points }
- Points are calculated automatically based on attendance, task completion, and engagement.

8. API Endpoints - Courses & Learning Track

Base path: /api/courses

GET /api/courses/ - List all courses

- Auth: Any logged-in user
- Returns courses with their parts and the user's progress on each part.

POST /api/courses/ - Create a course

- Auth: Requires 'add_course' permission
- Body: { title, description, image_url }

POST /api/courses/{course_id}/parts - Add a course part

- Auth: Requires 'add_course' permission
- Body: { title, content_type, content_url, order_index }
- IMPORTANT: order_index determines the sequential unlock order.

POST /api/courses/{course_id}/parts/{part_id}/complete - Mark part complete

- Auth: Any logged-in user
- Logic: The backend checks if the previous part (order_index - 1) is completed first.
- If ALL parts are completed, a certificate is automatically issued.

9. API Endpoints - Workshops

Base path: /api/workshops

GET /api/workshops/ - List all workshops

- Auth: Any logged-in user
- Returns workshops with session count and session details.

POST /api/workshops/ - Create a workshop

- Auth: Requires 'add_workshop' permission
- Body: { title, description, image_url, instructor }

POST /api/workshops/{id}/sessions - Add a workshop session

- Auth: Requires 'add_workshop' permission
- Body: { title, description, date, mode, location_or_link, materials_url }
- mode: 'online' or 'offline'

10. API Endpoints - Certificates

Base path: /api/certificates

GET /api/certificates/ - List my certificates

- Auth: Any logged-in user
- Returns all certificates earned by the current user.

POST /api/certificates/generate - Generate a certificate

- Auth: Any logged-in user
- Body: { source_type, source_id }
- source_type must be one of: 'course', 'workshop', 'event', 'session'

IMPORTANT VALIDATION RULES:

- course: ALL parts must be marked as 'completed' by the user.
- workshop: User must have at least one 'present' attendance record in the workshop's sessions.
- event: User must have 'attended' status in the event's attendance records.
- session: User must have 'attended' attendance_status in the session's scoring records.
- If validation fails, the server returns 400 with a descriptive error message.
- If the source_id does not exist, the server returns 404.

POST /api/certificates/manual - Manual certificate (Admin only)

- Auth: committee_admin, president, vice_president, secretary
- Body: { username, title }
- Bypasses all completion checks. Creates a custom certificate with any title for any user.

11. API Endpoints - Points & Leaderboard

Base path: /api/points

GET /api/points/leaderboard - Get points leaderboard

- Auth: Any logged-in user
- Returns all users ranked by total points (descending).
- Points are calculated as a live SUM from the point_entries table - never manually stored.

How Points Are Earned/Lost

Points are automatically created as ledger entries. The system never edits existing entries:

- Event attended: +2 points
- Event absent (excused): 0 points
- Event absent (unexcused): -1 point
- Session attendance/tasks/engagement: Variable (set by admin during scoring)
- Course completion: Points awarded via core_logic when all parts are done

12. API Endpoints - Feedback & Reports

Base path: /api/feedback and /api/reports

POST /api/feedback/ - Submit feedback

- Auth: Any logged-in user
- Body: { session_id, comment }

POST /api/feedback/{id}/report - Report a comment

- Auth: OC admin or higher
- Flags the comment for President/VP review. The comment stays visible.

DELETE /api/feedback/{id} - Delete reported feedback

- Auth: President or Vice President only
- CRITICAL: Deleting a reported comment also DELETES the author's user account (Rule 8).
- Both operations happen in a single atomic transaction.

13. API Endpoints - Dashboard & Access Control

Dashboard (/api/dashboard)

- GET /api/dashboard/stats - Returns aggregate statistics (total users, events, etc.)
- Auth: Admin roles only

Access Permissions (/api/access-permissions)

- GET /api/access-permissions/ - Returns the permission matrix for the current user.
- POST /api/access-permissions/ - Update permissions for a role. (President only)

Permissions are granular and control actions like: add_course, edit_course, delete_course, add_workshop, edit_workshop, delete_workshop, add_event, edit_event, delete_event, etc.

14. Certificate Generation Logic

The certificate generation flow is as follows:

- 1. The user (or admin) triggers certificate generation via the API.
- 2. The backend validates that the user has actually completed the source (course/workshop/event/session).
- 3. A personalized PNG image is generated using Pillow, with the user's full name and the title printed on a template.
- 4. The PNG is uploaded to the cloud (Cloudinary if configured, otherwise Google Drive).
- 5. The local PNG file is deleted after successful upload.
- 6. A Certificate record is saved to the database with the cloud URL as pdf_url.
- 7. The cloud URL is returned to the frontend for display/download.

Manual Certificates

Committee admins can bypass all completion checks and issue certificates with custom titles to any user by providing their username. This is useful for special recognition, participation awards, or cases where the standard automated flow does not apply.

15. Error Handling Guide

The backend uses standard HTTP status codes. Frontend developers should handle these:

Code	Status	Meaning
200	OK	Request successful
201	Created	Resource created successfully (new event, certificate, etc.)
204	No Content	Resource deleted successfully
400	Bad Request	Business logic failed (duplicate registration, incomplete course, etc.)
401	Unauthorized	Missing or invalid JWT token
403	Forbidden	User's role does not have permission for this action
404	Not Found	The requested resource ID does not exist
422	Validation	Request body failed schema validation (wrong field types)
500	Server Error	Unexpected backend error (check detail field for message)

16. Development Tips for Frontend

CORS Configuration

The backend currently allows all origins (`allow_origins=[*]`) for development. Before production, this should be restricted to the frontend's actual domain.

Date/Time Handling

All dates returned by the backend are standard ISO 8601 strings (e.g., '2026-09-17T19:37:38.427671'). Always parse them natively in JavaScript using `new Date()` or a library like `dayjs` before displaying.

File Downloads

Certificate URLs (`pdf_url`) are either cloud URLs (Cloudinary/Google Drive) or local static paths. If the URL starts with `/static/`, prepend the backend base URL. If it's a full `https://` URL, use it directly.

Pagination

Currently, list endpoints return all results without pagination. For large datasets, the frontend should implement client-side pagination or request backend pagination support.

Testing Workflow

- 1. Run `seed.py` to create admin accounts.
- 2. Login as 'president' to get full access.
- 3. Create events, courses, workshops, and sessions.
- 4. Register a member account and test the flow as a normal user.
- 5. Test certificate generation after completing courses/attending events.

Interactive API Docs

The easiest way to explore and test the API is the built-in Swagger UI at `http://localhost:8000/docs`. Click 'Authorize' at the top, enter your JWT token, and you can test any endpoint directly from your browser.